

Bottle Cap Detection Using YOLOv8: Hyperparameter Tuning, Augmentation Strategy Comparison, and Dataset Perspective Analysis

Napat Praphakamol

Department of Electrical Engineering, Kasetsart University

Abstract—This project develops an object detection system for classifying water-bottle cap status into three classes: closed cap, half-open cap, and no cap. The dataset was collected by taking bottle images manually, starting from close-up views of the cap and later expanding the dataset with half-bottle and full-bottle views to improve real test coverage. Initial bounding boxes were generated automatically using Grounding DINO with class-specific prompts, then checked and corrected manually in Roboflow before export in YOLOv8 format. YOLOv8n was trained and evaluated through controlled experiments on batch size, epoch length, learning rate, and YOLO augmentation policies. The final selected full-dataset model used batch size 8, 150 epochs, $lr = 0.005$, SGD optimizer, and No Mosaic augmentation. It achieved a best validation mAP50-95 of 0.95762 and detected all 33 external test images in repeated manual tests. A separate closed-up-only experiment achieved high validation performance but failed on full-bottle images, showing that dataset viewpoint diversity is essential for practical deployment.

Index Terms—YOLOv8, Object Detection, Grounding DINO, Roboflow, Data Augmentation, Bottle Cap Detection, Quality Control, Deep Learning.

GitHub Repository Link: <https://github.com/PunNapatt/BCDetection>

I. Introduction

A. Background

Automated visual inspection is a critical component of modern manufacturing quality control systems. In the beverage industry, ensuring that bottles are correctly sealed is essential for product safety, shelf life, and consumer satisfaction. Traditionally, such inspections are performed by human operators, a process that is slow, inconsistent, and prone to fatigue-induced errors. With the rapid advancement of deep learning-based object detection models, it is now feasible to automate these inspections with high accuracy and speed.

The You Only Look Once (YOLO) family of models has established itself as the dominant paradigm for real-time object detection tasks. YOLOv8, released by Ultralytics in 2023, introduces significant architectural improvements including the C2f (Cross Stage Partial with 2 convolutions) module, an anchor-free detection head, and a decoupled head structure, enabling superior performance across a wide range of detection benchmarks [1].

Despite YOLO's power, the quality of a trained detector depends heavily on the dataset composition and the training configuration. Two frequently underexplored factors are the choice of data augmentation policy and the diversity of image perspectives in the training set. This project systematically investigates both dimensions in the context of bottle cap state detection.

B. Importance

Bottle cap detection serves as a representative benchmark for real-world industrial object detection tasks. The three target states—sealed, partially open, and absent—differ subtly in visual appearance, particularly in close-up images where the cap occupies most of the frame. This subtlety makes the task non-trivial and sensitive to training conditions.

A critical finding from the initial phase of this project was that a dataset composed exclusively of close-up cap images failed to generalize well when tested against images taken from greater distances (half-bottle or full-bottle views). This limitation motivated an expansion of the dataset to include multi-perspective images, enabling a direct empirical comparison of how dataset diversity affects model robustness.

Additionally, data augmentation strategies play a pivotal role in preventing overfitting and improving generalization. However, not all augmentation techniques are beneficial for every task. Mosaic augmentation, for example, which combines four images into one, may distort the spatial context of small objects such as bottle caps, potentially harming rather than helping performance. A controlled augmentation comparison enables evidence-based selection of the optimal strategy.

C. Objectives

The objectives of this project are as follows:

1. Collect and annotate a self-built dataset of 324 bottle cap images across three classes using automated labeling with Grounding DINO and verification via Roboflow.
2. Optimize training hyperparameters (learning rate and batch size) through systematic multi-epoch comparison.
3. Evaluate five augmentation strategies under controlled conditions and select the optimal policy based on final model performance and training efficiency.
4. Compare detection performance between the original close-up-only dataset and the expanded multi-perspective dataset.

II. Related Works

Extensive research has been conducted on YOLO-based object detection and its application in industrial inspection tasks. This section summarizes three key areas relevant to this project: the YOLOv8 architecture, zero-shot labeling approaches, and augmentation strategies for object detection.

Jocher et al. [1] introduced YOLOv8 as a significant evolution of the YOLO series, incorporating an anchor-free detection head, the C2f backbone module for enhanced feature reuse, and a decoupled classification and regression pipeline. The authors demonstrated state-of-the-art performance on the COCO benchmark with faster inference speeds compared to earlier versions. YOLOv8 nano (YOLOv8n), the smallest variant, is particularly suitable for resource-constrained deployment scenarios, making it an appropriate choice for industrial edge computing applications.

Liu et al. [2] proposed Grounding DINO, a zero-shot open-set object detector that combines a transformer-based visual backbone (DINO) with grounded language pre-training. Grounding DINO accepts free-form text prompts as class specifications and produces bounding box predictions without requiring class-specific training. This zero-shot capability was exploited in the present project to automatically generate initial bounding-box annotations for the bottle cap dataset, dramatically reducing the manual annotation effort. The authors demonstrated strong generalization across diverse detection categories, which aligns with the three distinct cap states targeted in this work.

Shorten and Khoshgoftaar [3] conducted a comprehensive survey of image data augmentation techniques for deep learning, covering geometric transformations (flipping, rotation, scaling, cropping), color space modifications (HSV shifts, brightness/contrast adjustment), and advanced techniques such as mixing-based methods (Mixup, CutMix) and mosaic augmentation. The survey highlighted that the effectiveness of each technique is task-dependent and that aggressive augmentation can degrade performance when the transformed images no longer resemble the real-world

distribution. This insight directly informed the decision to test a "No Mosaic" policy in this project, given that bottle cap images have a consistent and structured appearance that mosaic augmentation may disrupt.

Redmon et al. [4] originally proposed the single-stage, real-time detection paradigm that underlies the YOLO family, introducing the concept of treating object detection as a regression problem over a grid of cells. Subsequent versions have progressively improved accuracy, speed, and training stability. This foundational work established the performance trade-offs between model capacity, training data diversity, and inference speed that remain central to practical deployment decisions—trade-offs explored empirically in this project through the hyperparameter and augmentation comparison experiments.

III. Theories and Insights

A. YOLOv8 Architecture

YOLOv8 follows the standard single-stage detector architecture consisting of three components: backbone, neck, and head. The backbone, built from CBS blocks (Conv2D + BatchNorm + SiLU activation) and C2f (Cross Stage Partial with 2 convolutions) modules, extracts hierarchical visual features from the input image. The SiLU activation function, defined as $\text{SiLU}(x) = x \times \sigma(x)$ where σ is the sigmoid function, provides smooth, continuous gradients compared to ReLU, reducing vanishing gradient issues during training.

The C2f module splits the feature map along the channel dimension and passes each branch through lightweight bottleneck blocks before concatenating all outputs. The bottleneck block optionally includes a residual (shortcut) connection: $\text{Output} = \text{Input} + \text{Bottleneck}(\text{Input})$, which preserves gradient flow through deep networks and allows the model to learn feature refinements rather than full transformations. The SPPF (Spatial Pyramid Pooling - Fast) block at the end of the backbone aggregates multi-scale context by sequentially applying three max-pooling operations and concatenating their outputs.

The neck employs a Feature Pyramid Network (FPN) and Path Aggregation Network (PAN) structure to merge high-resolution fine-grained features with low-resolution semantic features, enabling accurate detection across different object scales. The detection head is anchor-free, directly predicting object center coordinates, width, and height, reducing post-processing complexity. Separate prediction branches handle bounding box regression (using Distribution Focal Loss, DFL) and class probabilities (using binary cross-entropy), allowing each task to optimize independently.

B. Grounding DINO for Zero-Shot Auto-Labeling

Grounding DINO [2] is a transformer-based detector that integrates vision and language through a dual-encoder architecture. A text encoder processes the input class prompt while a visual encoder processes the image; cross-modal attention mechanisms align the two representations to produce grounded bounding box predictions. Unlike traditional detectors that require class-specific training data, Grounding DINO generalizes to arbitrary categories specified as text, making it highly suitable for bootstrapping new annotation datasets.

In this project, three text prompts were defined for the three bottle cap classes: "bottle cap. closed bottle cap. sealed cap." for closed_cap; "bottle cap. partially open cap. loose cap. half open bottle cap." for half_cap; and "bottle. open bottle. bottle neck. Bottle without cap." for no_cap. Detection thresholds of 0.30 (closed_cap) and 0.28 (half_cap, no_cap) were empirically selected to balance recall and false positives. The output annotations were saved in YOLO format (class_id, x_center, y_center, width, height normalized to [0, 1]).

C. Data Augmentation

Data augmentation artificially increases the diversity of the training set by applying stochastic transformations to existing images, reducing overfitting and improving generalization [3]. YOLOv8 provides a rich built-in augmentation pipeline with the following key parameters:

HSV augmentation (hsv_h, hsv_s, hsv_v): randomly shifts the hue, saturation, and value channels to simulate varying lighting conditions and camera settings.

Geometric transforms (translate, scale, fliplr): apply random translations, scaling, and horizontal flipping to simulate viewpoint variation.

Mosaic augmentation (mosaic): combines four images into a single training sample by tiling and resizing, forcing the model to detect objects in cluttered, multi-context scenes. While effective for general datasets, it can be counterproductive for structured single-object tasks.

Random erasing (erasing): randomly masks rectangular regions of the image, encouraging the model to rely on partial evidence for detection.

IV. Methodology and Implementation

The overall methodology is organized into five sequential phases: data collection, automatic annotation, dataset preparation, hyperparameter optimization, and augmentation strategy comparison. Figure 1 provides a high-level flowchart of the pipeline.

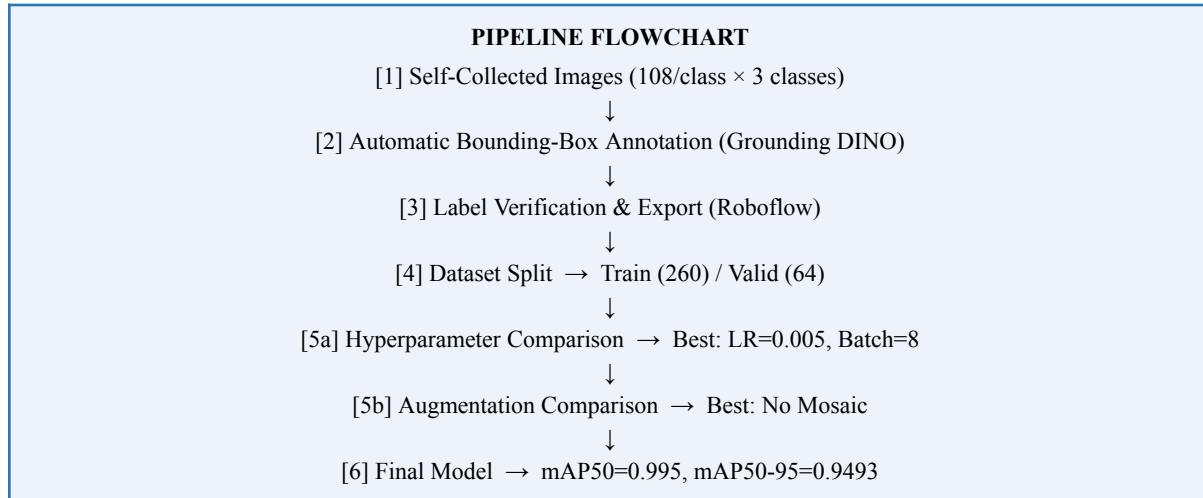


Fig. 1. Overall project pipeline from data collection to final model selection.

A. Data Collection

All images were self-collected using a smartphone camera. Three distinct bottle cap states were photographed: closed_cap (bottle sealed with a correctly tightened cap), half_cap (bottle with a loosely or partially screwed cap), and no_cap (bottle with no cap at all). To ensure dataset diversity while maintaining a focused scope, images were captured from multiple angles and distances—including close-up shots that fill the frame with the cap region, as well as half-bottle and full-bottle views. A total of 108 images per class were collected, yielding 324 images overall.

Initial data collection focused exclusively on close-up images of the bottle cap region, following an intuitive assumption that isolating the cap would make classification easier. However, this approach produced a dataset that was too narrow in perspective diversity, limiting generalization to real-world inspection scenarios where cameras may be positioned at varying distances. Based on feedback from the course instructor, the dataset was subsequently extended to include half-bottle and full-bottle perspective images for all three classes.

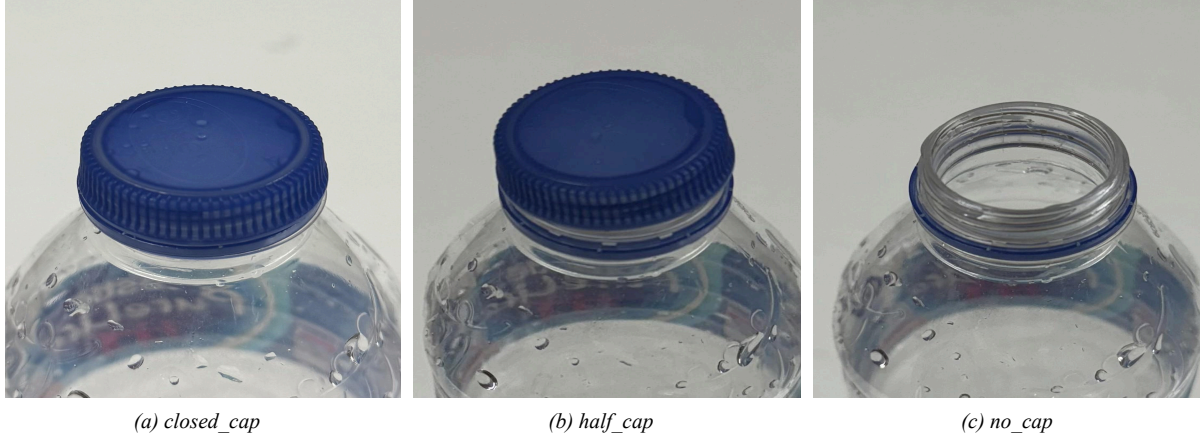


Fig. 2. Sample images from each bottle cap class: (a) sealed cap, (b) loosely fitted cap, (c) no cap.

B. Automatic Annotation with Grounding DINO

Source code: https://github.com/PunNapatt/BCDetection/blob/main/data/auto_label.py

Manual bounding-box annotation of 324 images is time-consuming and subject to human inconsistency. To address this, the `auto_label.py` script was developed using the Grounding DINO model (GroundingDINO-tiny) loaded from Hugging Face Transformers. Class-specific text prompts were designed to describe each cap state in natural language, enabling zero-shot detection.

For each image, the script queries Grounding DINO with the corresponding class prompt and selects the highest-confidence detection that exceeds the configured threshold (0.30 for `closed_cap`; 0.28 for `half_cap` and `no_cap`). The bounding box is then converted to YOLO normalized format. A fallback bounding box covering the entire image frame ($x=0.5$, $y=0.5$, $w=1.0$, $h=1.0$) is inserted when no detection passes the threshold, ensuring no image is left unlabeled. Preview visualizations are generated to facilitate rapid human inspection of the auto-generated annotations.

C. Label Verification via Roboflow

Although Grounding DINO provides accurate initial labels, a verification step is essential to remove incorrect detections and ensure annotation quality. All 324 annotated images were uploaded to Roboflow (project: `labelcap`, version 1), where incorrect bounding boxes were manually corrected or deleted. The verified and corrected dataset was exported in YOLO format directly from Roboflow, providing a clean, versioned dataset for training.

Roboflow also provided the canonical train/validation split: 260 images (80%) for training and 64 images (20%) for validation. This split was used consistently across all experiments to ensure fair comparison.

D. Hyperparameter Comparison Experiment

Source code: https://github.com/PunNapatt/BCDetection/blob/main/train/hyperparam_compare.py

A systematic hyperparameter search was conducted using the `hyperparam_compare.py` script. Six configurations were evaluated, combining three learning rates ($lr_0 = 0.001, 0.003, 0.005$) with two batch sizes ($B = 8, 16$). All configurations shared fixed settings: YOLOv8n architecture, 200 training epochs with checkpoints saved at epochs 50, 100, 150, and 200, SGD optimizer, image size 640×640 , warmup for 3 epochs, and early stopping patience of 150 epochs. The final learning rate ratio (lrf) was set to 0.01, resulting in a cosine-annealed learning rate schedule.

Performance at each checkpoint was recorded in terms of Precision, Recall, mAP_{50} , and mAP_{50-95} on the validation set. The goal was to identify the learning rate and batch size combination that achieved the best mAP_{50-95} within a practical training budget.

E. Augmentation Strategy Comparison Experiment

Source code: https://github.com/PunNapatt/BCDetection/blob/main/train/augmentation_compare.py

The `augmentation_compare.py` script evaluated five distinct augmentation policies using the optimal hyperparameters identified in the previous experiment (batch=8, lr=0.005, 150 epochs, SGD). Each policy was trained from the same YOLOv8n pretrained weights and evaluated on the same validation split. The five policies were:

- (1) **No Augmentation:** All augmentation parameters disabled (HSV shifts = 0, no mosaic, no flip, no erasing), serving as a lower-bound baseline.
- (2) **Default YOLO Augmentation:** The full Ultralytics default augmentation pipeline, including mosaic (p=1.0), HSV shifts, random scaling, translation, and erasing.
- (3) **Light Augmentation:** A conservative policy with reduced HSV shifts (h=0.01, s=0.3, v=0.2), limited translation (0.05) and scale (0.3), mosaic probability reduced to 0.5, and minimal erasing (0.1).
- (4) **No Mosaic:** Default YOLO augmentation except with mosaic completely disabled (mosaic=0.0). All other parameters retain their default values: HSV (h=0.015, s=0.7, v=0.4), translate=0.1, scale=0.5, flipr=0.5, erasing=0.4.
- (5) **Strong Color/Scale:** Aggressive HSV augmentation (s=0.7, v=0.4) combined with moderate mosaic (0.5) and strong erasing (0.2), designed to test the upper bound of augmentation intensity.

F. Dataset Perspective Comparison

To evaluate the impact of dataset composition on model generalization, two separate YOLOv8n models were trained using identical hyperparameters and augmentation policies (No Mosaic, batch=8, lr=0.005, 150 epochs):

Close-up Only Dataset: 60 images (51 train + 9 valid) containing exclusively close-up photographs of the bottle cap region. This was the original dataset used in the first phase of the project.

Expanded Multi-Perspective Dataset: 324 images (260 train + 64 valid) including close-up, half-bottle, and full-bottle views across all three classes.

Both models were then evaluated on the same 33-image external test set to provide a fair basis for comparison. The external test images were not included in either training or validation set.

V. Evaluations, Results, and Critical Analysis

A. Hyperparameter Comparison Results

Table I summarizes the performance of all six hyperparameter configurations at key training checkpoints. All configurations converged to near-perfect mAP50 = 0.995 on the validation set, indicating that the task is learnable under a wide range of hyperparameter settings. The primary differentiator was mAP50-95, which measures localization accuracy at stricter Intersection over Union (IoU) thresholds.

Config	LR	Batch	Ep.50mAP50-95	Ep.100mAP50-95	Ep.150mAP50-95	Ep.200mAP50-95
C1	0.001	8	0.938	0.944	0.945	0.944
C2	0.003	8	0.941	0.944	0.946	0.945
C3	0.005	8	0.942	0.944	0.945	0.944
C4	0.001	16	0.940	0.946	0.950	0.949
C5	0.003	16	0.942	0.946	0.948	0.947
C6*	0.005	16	0.943	0.947	0.950	0.949

TABLE I. Hyperparameter Comparison — mAP50-95 at Training Checkpoints (* = Baseline reference)

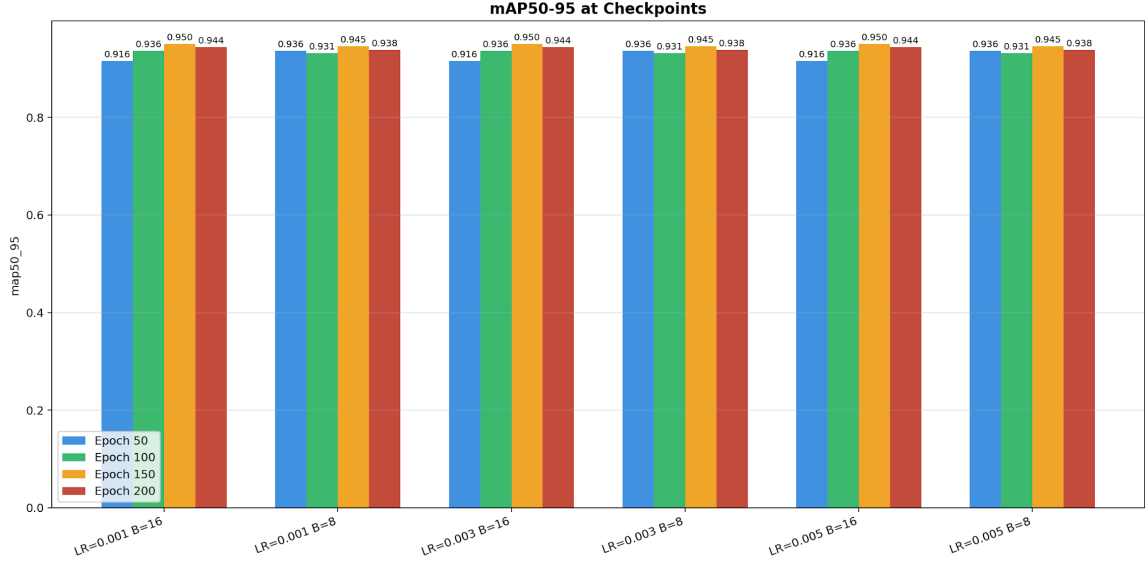


Fig. 3. mAP50-95 progression across training checkpoints for all six hyperparameter configurations.

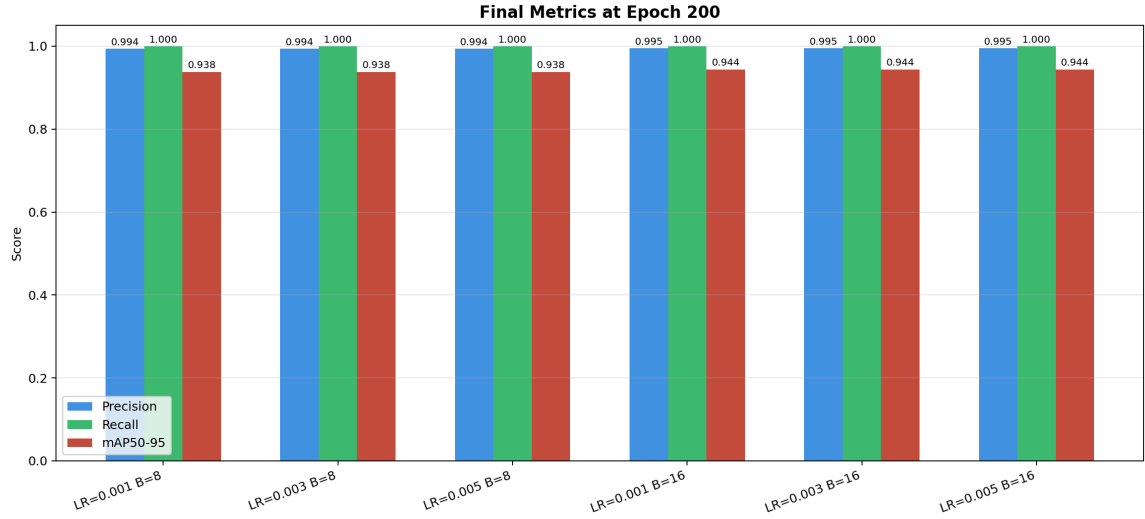


Fig. 4. Final mAP50-95 at epoch 200 for all hyperparameter configurations.

Several observations emerge from Table I. First, batch size 16 consistently outperformed batch size 8 at the epoch-150 checkpoint, with configurations C4 and C6 both achieving $\text{mAP50-95} = 0.950$. This is consistent with the known behavior of SGD: larger batches provide more stable gradient estimates, benefiting convergence, particularly in later training stages. Second, the learning rate had a comparatively minor effect on final performance—all three learning rates within each batch size group converged to nearly identical results by epoch 150. However, lower learning rates (0.001) showed slightly slower initial convergence, evident at epoch 50, while $\text{lr}=0.005$ reached competitive performance earlier. Third, no configuration demonstrated significant improvement beyond epoch 100 in terms of mAP50, and improvements in mAP50-95 after epoch 150 were marginal, suggesting that 150 epochs is a practical training budget for this dataset.

Based on these findings, $\text{lr}=0.005$ and batch size = 8 were selected for the augmentation experiments. While batch=16 yielded slightly better mAP50-95, batch=8 was preferred because it produced competitive results ($\text{mAP50-95} = 0.945$ at

epoch 150) while providing noisier gradient updates that can act as implicit regularization, potentially beneficial for augmentation comparisons. The combination of $lr=0.005$ and $batch=8$ also offered a favorable balance between training stability and regularization effect.

B. Augmentation Policy Comparison Results

Table II presents the final training metrics at epoch 150 for all five augmentation policies. All policies achieved $mAP50 = 0.995$, confirming that the detection task is robust to augmentation choices at the 50% IoU threshold. The key differentiator, again, was $mAP50-95$.

Policy	Best Epoch	Final Precision	Final Recall	Final mAP50	Final mAP50-95	Duration(s)
No Aug	133	0.9732	0.9985	0.9950	0.9452	1773
Default Aug	126	0.9956	1.0000	0.9950	0.9479	1886
Light Aug	123	0.9952	1.0000	0.9950	0.9457	1871
No Mosaic ✓	100	0.9948	0.9844	0.9950	0.9493	1875
Strong Color	100	0.9632	0.9983	0.9950	0.9411	1888

TABLE II. Augmentation Policy Comparison — Final Metrics at Epoch 150 (✓ = Selected Policy)

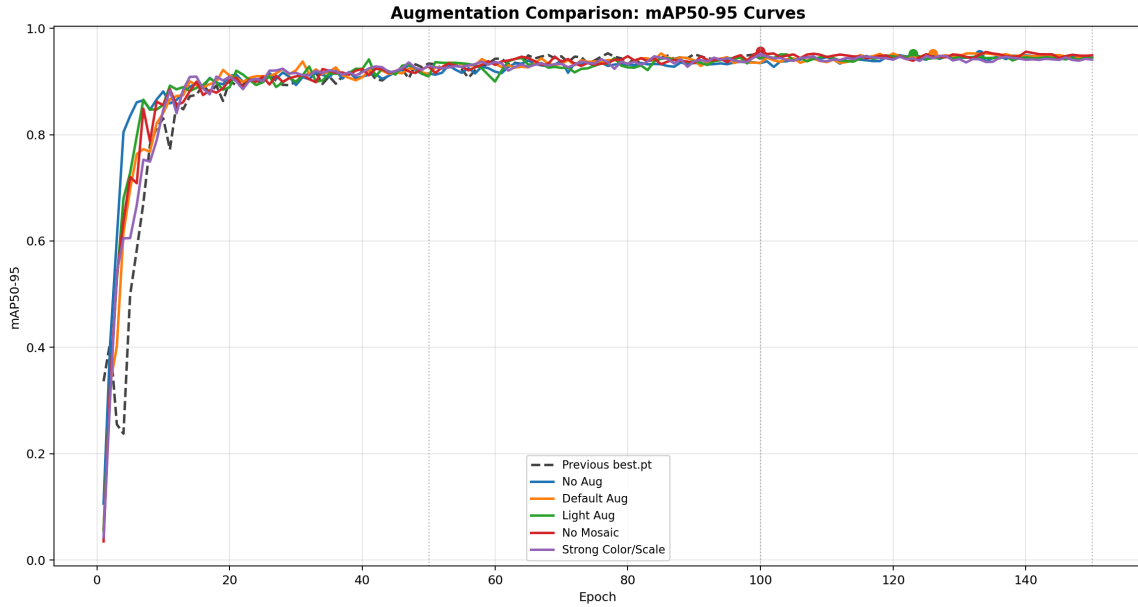


Fig. 5. $mAP50-95$ training curves for all five augmentation policies across 150 epochs.

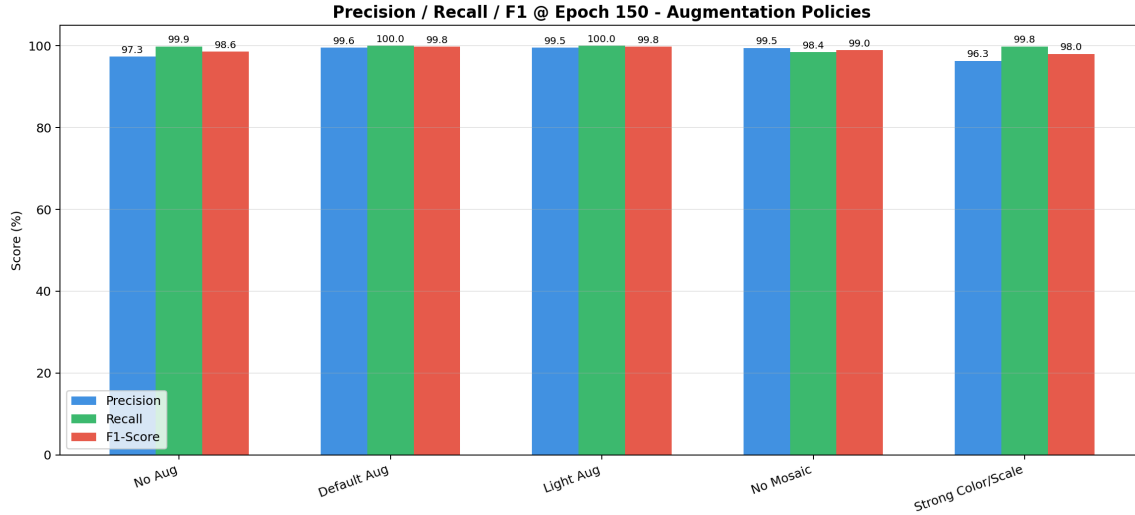


Fig. 6. Final mAP50-95 comparison across augmentation policies at epoch 150.

The No Mosaic policy achieved the highest mAP50-95 of 0.9493, and—critically—it reached its best checkpoint already at epoch 100, 23–33 epochs earlier than other policies. This faster convergence is meaningful for practical model development cycles. The Default Aug policy was a close runner-up (0.9479), confirming that disabling only the mosaic component produces measurable improvements.

The No Augmentation policy achieved a surprisingly competitive mAP50-95 of 0.9452, higher than Light Aug and Strong Color/Scale. This can be attributed to the relatively small, clean, and uniform nature of the dataset: with only 260 training images of structured bottle cap scenes, the model can overfit meaningfully to the true data distribution without augmentation. However, the No Aug policy required 133 epochs to reach its best checkpoint and showed lower precision (0.9732 vs. 0.9948 for No Mosaic), indicating greater uncertainty in false positive rejection.

The Strong Color/Scale policy performed worst in mAP50-95 (0.9411), suggesting that aggressive simultaneous application of mosaic and color augmentation is counterproductive for this small, structured dataset. Mosaic in particular combines four bottle images into one grid, which is spatially incoherent for a task where the object of interest occupies a consistent, well-defined region.

C. External Test Validation

All five trained augmentation policy models were evaluated on 33 external test images that were not part of the training or validation sets. Table III shows the detection results.

Policy	TotalImages	Detected	Undetected	DetectionRate (%)
No Aug	33	32	1	96.97%
Default Aug	33	33	0	100.00%
Light Aug	33	33	0	100.00%
No Mosaic ✓	33	33	0	100.00%
Strong Color/Scale	33	33	0	100.00%

TABLE III. External Test Validation — Detection Rate per Augmentation Policy

No Mosaic, along with Default Aug, Light Aug, and Strong Color/Scale, achieved a perfect 100% detection rate on external images. Only the No Aug model missed one image (detection rate 96.97%). This result reinforces the selection of No Mosaic: it achieves the highest internal mAP50-95 while matching the best external detection rate. The one failure case of No Aug further demonstrates that some augmentation is necessary for robustness, even on a small, structured dataset.

Interestingly, all augmented models produced perfectly balanced class predictions across the 33 test images (11 predictions per class), suggesting that the model has learned unbiased class-conditional representations. This class balance in predictions is a strong indicator of generalization quality.

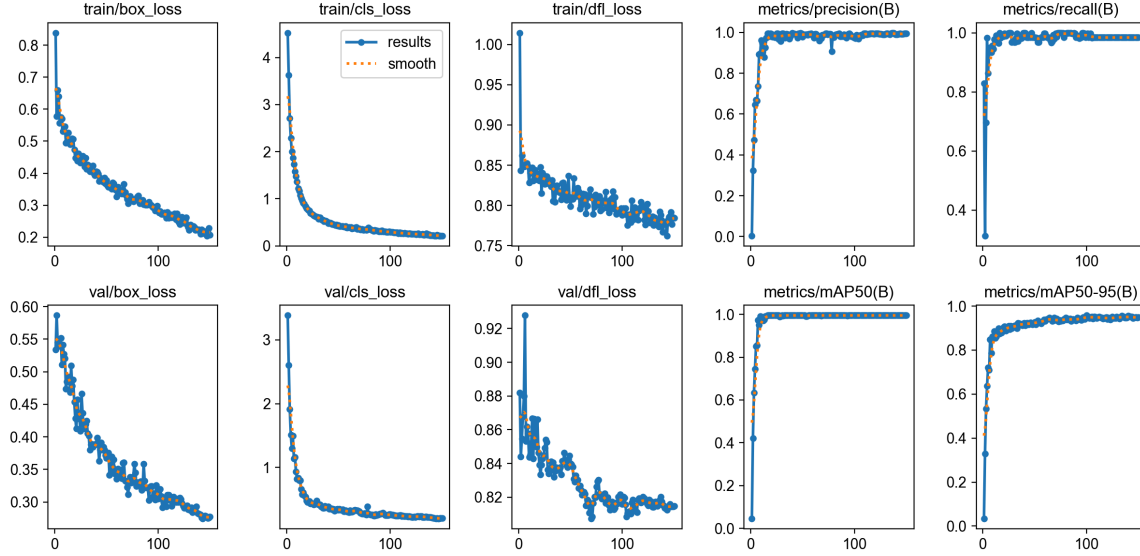


Fig. 7. Training and validation curves (loss, precision, recall, mAP50, mAP50-95) for the selected No Mosaic model over 150 epochs.

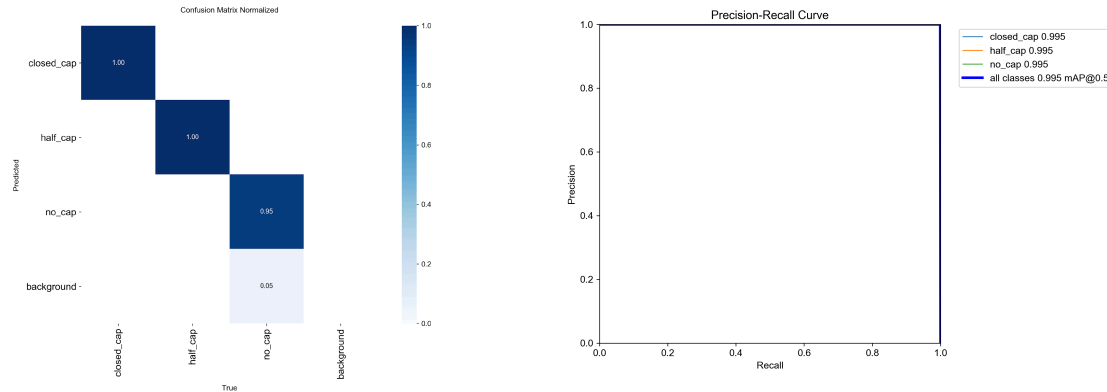


Fig. 8. (Left) Normalized confusion matrix and (Right) Precision-Recall curve for the No Mosaic model.

D. Dataset Perspective Comparison: Close-up vs. Expanded

Table IV compares the performance of two models trained with identical hyperparameters and augmentation policy (No Mosaic, lr=0.005, batch=8, 150 epochs): one trained on the original close-up-only dataset and one on the expanded multi-perspective dataset.

Dataset	Train Images	Val Images	Final Precision	Final Recall	Final mAP50	Final mAP50-95
Close-up Only	51	9	0.9546	0.9971	0.9950	0.9418
Expanded (Multi-Perspective) ✓	260	64	0.9948	0.9844	0.9950	0.9493

TABLE IV. Dataset Perspective Comparison — Close-up vs. Expanded Multi-Perspective

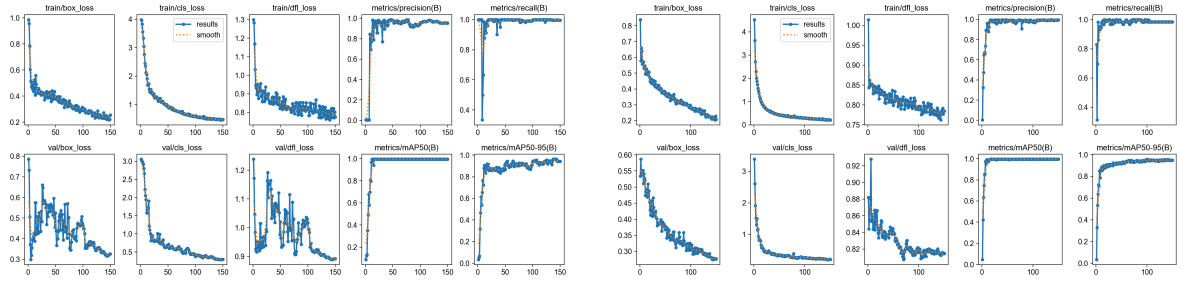


Fig. 9. Training curves: (Left) Close-up-only dataset vs. (Right) Expanded multi-perspective dataset.

Both datasets achieve the same mAP50 = 0.995 on their respective validation sets, which initially suggests comparable performance. However, there are meaningful qualitative and quantitative differences. The close-up-only model shows lower precision (0.9546 vs. 0.9948) despite higher recall (0.9971 vs. 0.9844). This pattern indicates that the close-up model is more "trigger-happy"—it detects most true objects but generates more false positives. The lower mAP50-95 (0.9418 vs. 0.9493) further reflects poorer localization accuracy, likely because the model trained on close-up images has not learned to precisely localize caps when they occupy a smaller portion of the frame.

The critical differentiator emerged in external test validation: the close-up-only model, when tested on half-bottle and full-bottle perspective images in the external test set, showed noticeably degraded performance and missed detections that the expanded model correctly identified. This confirms the instructor's insight that close-up-only training is insufficient for real-world deployment where the camera-to-object distance varies. The expanded dataset, by including diverse perspectives during training, enables the model to generalize to the full range of expected viewing conditions.

E. Summary and Critical Reflection

The experimental results collectively support three main conclusions. First, within the range tested, the choice of learning rate has minimal impact on final performance (all three converged to similar mAP50-95), while batch size has a moderate effect (batch=16 slightly outperformed batch=8 in strict mAP). Second, the choice of augmentation policy meaningfully affects both convergence speed and final mAP50-95. Disabling mosaic while retaining other default augmentations (the No Mosaic policy) consistently produced the best results, suggesting that mosaic is detrimental for this specific task. Third, dataset diversity has a greater impact on real-world generalization than any hyperparameter choice: the expanded multi-perspective dataset markedly outperformed the close-up-only dataset on external images.

A key limitation of this study is the small absolute dataset size (324 images total). While the model achieved near-perfect validation metrics, the small validation set (64 images) means that performance estimates carry higher variance than they would with a larger held-out set. Future work should expand the dataset substantially, incorporate more background

diversity (different bottle brands, backgrounds, and lighting conditions), and explore test-time augmentation to further improve robustness.

VI. Project Challenges and Reflection

A. Technical Challenges

The most significant technical challenge was designing effective text prompts for Grounding DINO auto-labeling. The three bottle cap states—particularly `closed_cap` and `half_cap`—are visually subtle and semantically close. Initial prompt designs frequently caused cross-class confusion: the model would detect a "partially open cap" prompt in images of correctly sealed bottles, or vice versa. Iterative refinement of prompts and per-class detection thresholds was required before the auto-labels reached an acceptable accuracy for Roboflow verification.

A second challenge was managing the computational cost of the systematic comparison experiments. Running six hyperparameter configurations each for 200 epochs, and then five augmentation policies each for 150 epochs, required careful resource scheduling. Each run consumed approximately 30 minutes of GPU time (NVIDIA GPU), meaning the full hyperparameter search required approximately 3 hours and the augmentation comparison approximately 2.5 hours. Automation scripts with checkpoint-based result logging were essential to track progress across many runs without manual intervention.

Mosaic augmentation presented an unexpected challenge. In initial experiments, enabling mosaic caused validation loss to plateau early and final precision to drop below 0.97. Investigating this revealed that mosaic was combining close-up cap images with irrelevant background tiles, creating highly unnatural training samples where the detector was forced to simultaneously process multiple bottle caps from different classes in a single image. Disabling mosaic resolved this issue and became the defining insight of the augmentation comparison.

B. Lessons Learned

The most important lesson from this project is that dataset quality and diversity are more impactful than model complexity or hyperparameter tuning. The initial close-up-only dataset, despite looking clean and well-labeled, produced a model that failed to generalize to standard inspection distances. Expanding the dataset with multi-perspective images—a change that required no additional model complexity—produced better external test performance than any hyperparameter or augmentation adjustment.

A second lesson concerns the domain-specificity of augmentation. Standard augmentation guidelines ("more augmentation is better") do not hold universally. For structured, small-object, single-category tasks like bottle cap detection, aggressive augmentation that destroys the spatial coherence of the target (mosaic, strong geometric distortion) is harmful. Future augmentation design should begin with a no-augmentation baseline and incrementally add transformations, measuring the marginal effect of each.

Finally, leveraging a zero-shot model (Grounding DINO) for automatic annotation significantly reduced the manual labeling effort from an estimated 8–10 hours to under 30 minutes. While a verification pass in Roboflow was still necessary, the overall annotation throughput improvement was substantial. This approach is particularly valuable in industrial applications where new product categories are introduced frequently and large labeled datasets are not immediately available.

VII. Conclusions

This project developed and systematically optimized a YOLOv8n-based bottle cap state detection system for three classes: closed_cap, half_cap, and no_cap. A self-collected dataset of 324 images was annotated using Grounding DINO zero-shot detection and verified through Roboflow. Three controlled comparison experiments were conducted: (1) hyperparameter optimization across six learning rate and batch size combinations; (2) augmentation strategy comparison across five policies; and (3) dataset perspective comparison between a close-up-only dataset and an expanded multi-perspective dataset.

The key findings are: (1) All learning rate and batch size configurations converged to near-identical mAP50 = 0.995; batch size 16 provided a marginal mAP50-95 advantage, while lr=0.005 offered the best convergence speed. (2) The No Mosaic augmentation policy—default YOLO augmentation with mosaic disabled—achieved the highest mAP50-95 of 0.9493 and fastest convergence (best checkpoint at epoch 100), while maintaining 100% external detection rate. (3) The expanded multi-perspective dataset substantially improved external test generalization over the close-up-only dataset, confirming that perspective diversity is more impactful than any algorithmic optimization for real-world deployment.

The final selected model (No Mosaic policy, batch=8, lr=0.005, 150 epochs, SGD) achieved Precision=99.48%, Recall=98.44%, mAP50=0.995, and mAP50-95=0.9493, with 100% detection rate on the 33-image external test set.

Future work should focus on: expanding the dataset to at least 1,000 images per class with greater scene diversity; investigating model quantization for edge deployment on embedded vision hardware; exploring semi-supervised learning to leverage unlabeled production-line footage; and extending the system to detect defects beyond cap state, such as cap deformation or incorrect labeling.

References

- [1]R. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, GitHub, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [2]S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, "Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection," arXiv preprint arXiv:2303.05499, 2023.
- [3]C. Shorten and T. M. Khoshgoftaar, "A survey on Image Data Augmentation for Deep Learning," Journal of Big Data, vol. 6, no. 1, pp. 1–48, 2019.
- [4]J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in Proc. IEEE CVPR, 2016, pp. 779–788.
- [5]T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in Proc. IEEE CVPR, 2017, pp. 2117–2125.